Working with Pulseq in Matlab

Andreia S Gaspar

Instituto Superior Técnico,

Universidade de Lisboa, Portugal

andreia.gaspar@tecnico.ulisboa.pt

February 2026

Pulseq: Overview

Pulseq

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

Python

pypulseq

Ravi K, et al. *JOSS*. 2019

https://github.com/imr-framework/pypulseq

Jemris

https://www.jemris.org/

Pulseq file (*.seq)

Interpreter

Read the sequence ***.seq file** and translate it into the **specific MR system execution**

Currently

- Siemens
- GE
- Bruker
- Phillips

Pulseq: Overview

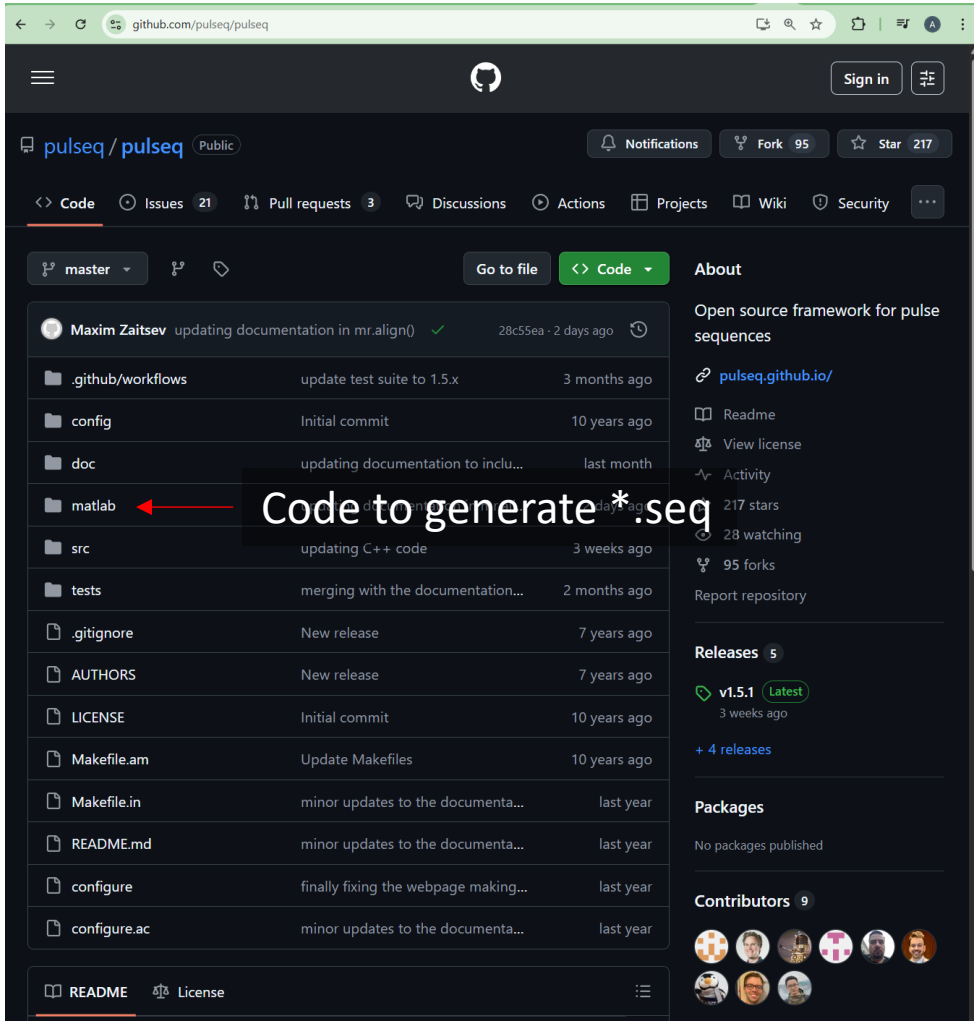
Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io



Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

Python

pypulseq

Ravi K, et al. *JOSS*. 2019

https://github.com/imr-framework/pypulseq

Jemris

https://www.jemris.org/

Pulseq file (*.seq)

Blocks

	duration	rf	gx	gy	gz	adc	ext
1	42	1	0	0	0	0	0
2	500	0	0	0	0	0	0
3	10244	0	0	0	0	1	0

Events

		amplitude	mag	phase	time_id	center	delay	ppm_	freq_	ph_off.	use
RF	1	833.3	1	2	0	150	100	0	0	0	e

		num	dwell	delay	ppm_	freq_	ph_off.	ph_id
ADC	1	1024	100000	20	0	0	0	0

Shapes

id	1
num	300

1
0
0
297

compressed shape data

Check: https://pulseq.github.io/specification.pdf

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

```
                adcDeadTime: 1.0000e-05
        bl
                addGradients
                addRamps
                align
                aux
                block2events
                calcAdcSeg
                calcDuration
                calcRamp
fx >> mr.
```

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

1. Hardware limits:

```
sys = mr.opts('MaxGrad', 28, 'GradUnit',
'mT/m','MaxSlew', 120, 'SlewUnit',
'T/m/s', 'rfRingdownTime', 20e-
6,  'rfDeadTime', 100e-6, 'adcDeadTime',
10e-6);
```

Command Window

```
>> sys = mr.opts('MaxGrad', 28, 'GradUnit', 'mT/m', ...
    'MaxSlew', 120, 'SlewUnit', 'T/m/s', 'rfRingdownTime', 20e-6, ...
    'rfDeadTime', 100e-6, 'adcDeadTime', 10e-6);
>> sys

sys =

  struct with fields:

                 maxGrad: 1192128
                 maxSlew: 5.1091e+09
                   maxB1: 851.5200
                riseTime: []
              rfDeadTime: 1.0000e-04
          rfRingdownTime: 2.0000e-05
             adcDeadTime: 1.0000e-05
           adcRasterTime: 1.0000e-07
            rfRasterTime: 1.0000e-06
           gradRasterTime: 1.0000e-05
      blockDurationRaster: 1.0000e-05
          adcSamplesLimit: 0
           rfSamplesLimit: 0
         adcSamplesDivisor: 4
                   gamma: 42576000
                      B0: 1.5000

fx >>
```

Find scanner specific hardware limits

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

1. Hardware limits:

```
sys = mr.opts('MaxGrad', 28, 'GradUnit',
'mT/m','MaxSlew', 120, 'SlewUnit',
'T/m/s', 'rfRingdownTime', 20e-
6,  'rfDeadTime', 100e-6, 'adcDeadTime',
10e-6);
```

2. Create sequence object

```
seq=mr.Sequence(sys);
```

```
>> seq

seq =

  Sequence with properties:

       version_major: 1
       version_minor: 5
    version_revision: 1
        rfRasterTime: 1.0000e-06
      gradRasterTime: 1.0000e-05
       adcRasterTime: 1.0000e-07
 blockDurationRaster: 1.0000e-05
         definitions: [4×1 containers.Map]
          blockEvents: {}
       blockDurations: []
            rfLibrary: [1×1 mr.EventLibrary]
          gradLibrary: [1×1 mr.EventLibrary]
           adcLibrary: [1×1 mr.EventLibrary]
          trigLibrary: [1×1 mr.EventLibrary]
      labelsetLibrary: [1×1 mr.EventLibrary]
      labelincLibrary: [1×1 mr.EventLibrary]
     extensionLibrary: [1×1 mr.EventLibrary]
         shapeLibrary: [1×1 mr.EventLibrary]
        rfShimLibrary: [1×1 mr.EventLibrary]
     softDelayLibrary: [1×1 mr.EventLibrary]
      softDelayHints1: [0×1 containers.Map]
      softDelayHints2: {}
      rotationLibrary: [1×1 mr.EventLibrary]
   extensionStringIDs: {}
  extensionNumericIDs: []
        gradCheckData: [1×1 struct]
        signatureType: ''
        signatureFile: ''
       signatureValue: ''
         rfID2NameMap: [0×1 containers.Map]
        adcID2NameMap: [0×1 containers.Map]
       gradID2NameMap: [0×1 containers.Map]
                  sys: [1×1 struct]

fx >>
```

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

1. Hardware limits:

```
sys = mr.opts('MaxGrad', 28, 'GradUnit',
'mT/m','MaxSlew', 120, 'SlewUnit',
'T/m/s', 'rfRingdownTime', 20e-
6,  'rfDeadTime', 100e-6, 'adcDeadTime',
10e-6);
```

2. Create sequence object

```
seq=mr.Sequence(sys);
```

```
>> seq

seq =

  Sequence with properties:

          version_major: 1
          version_minor: 5
       version_revision: 1
           rfRasterTime: 1.0000e-06
         gradRasterTime: 1.0000e-05
          adcRasterTime: 1.0000e-07
     blockDurationRaster: 1.0000e-05
            definitions: [4×1 containers.Map]
             blockEvents: {}
          blockDurations: []
               rfLibrary: [1×1 mr.EventLibrary]
             gradLibrary: [1×1 mr.EventLibrary]
              adcLibrary: [1×1 mr.EventLibrary]
             trigLibrary: [1×1 mr.EventLibrary]
          labelsetLibrary: [1×1 mr.EventLibrary]
          labelincLibrary: [1×1 mr.EventLibrary]
         extensionLibrary: [1×1 mr.EventLibrary]
             shapeLibrary: [1×1 mr.EventLibrary]
            rfShimLibrary: [1×1 mr.EventLibrary]
         softDelayLibrary: [1×1 mr.EventLibrary]
          softDelayHints1: [0×1 containers.Map]
          softDelayHints2: {}
          rotationLibrary: [1×1 mr.EventLibrary]
       extensionStringIDs: {}
      extensionNumericIDs: []
            gradCheckData: [1×1 struct]
            signatureType: ''
            signatureFile: ''
           signatureValue: ''
             rfID2NameMap: [0×1 containers.Map]
            adcID2NameMap: [0×1 containers.Map]
           gradID2NameMap: [0×1 containers.Map]
                      sys: [1×1 struct]

fx >>
```

No blocks yet

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

1. Hardware limits:

```
sys = mr.opts('MaxGrad', 28, 'GradUnit',
'mT/m','MaxSlew', 120, 'SlewUnit',
'T/m/s', 'rfRingdownTime', 20e-
6,  'rfDeadTime', 100e-6, 'adcDeadTime',
10e-6);
```

2. Create sequence object

```
seq=mr.Sequence(sys);
```

3. Generate elements (e.g. RF, gz)

```
[rf, gz] = mr.makeSincPulse(FlipAngle,
'Duration', 4e-3, 'SliceThickness',
sliceThickness, 'apodization',0.5,
'timeBwProduct', 4, 'system',sys);
```

Command Window

```
>> [rf, gz] = mr.makeSincPulse(FlipAngle,'Duration',490*1e-6,...
              'SliceThickness',6e-3,'timeBwProduct',1.5,...
              'system',sys, 'use', 'excitation');
>> rf

rf =

  struct with fields:

            type: 'rf'
          signal: [490×1 double]
               t: [490×1 double]
       shape_dur: 4.9000e-04
      freqOffset: 0
     phaseOffset: 0
         freqPPM: 0
        phasePPM: 0
        deadTime: 1.0000e-04
    ringdownTime: 2.0000e-05
           delay: 1.0000e-04
             use: 'excitation'
          center: 2.4500e-04

>> gz

gz =

  struct with fields:

         type: 'trap'
      channel: 'z'
    amplitude: 5.1020e+05
     riseTime: 1.0000e-04
     flatTime: 4.9000e-04
     fallTime: 1.0000e-04
         area: 301.0204
     flatArea: 250
        delay: 0
        first: 0
         last: 0
```

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

1. Hardware limits:

```
sys = mr.opts('MaxGrad', 28, 'GradUnit',
'mT/m','MaxSlew', 120, 'SlewUnit',
'T/m/s', 'rfRingdownTime', 20e-
6,  'rfDeadTime', 100e-6, 'adcDeadTime',
10e-6);
```

2. Create sequence object

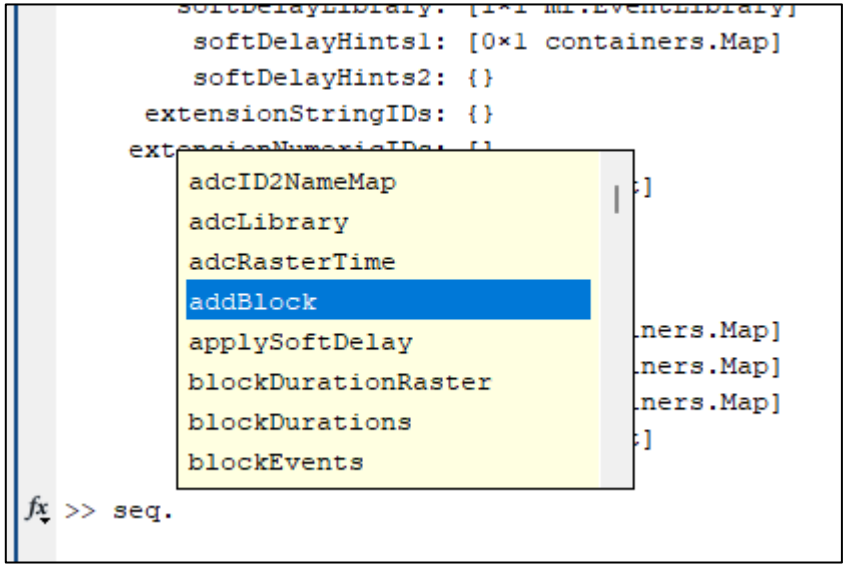
```
seq=mr.Sequence(sys);
```

3. Generate elements (e.g. RF, gz)

```
[rf, gz] = mr.makeSincPulse(FlipAngle,
'Duration', 4e-3, 'SliceThickness',
sliceThickness, 'apodization',0.5,
'timeBwProduct', 4, 'system',sys);
```

4. Add blocks (e.g. Block 1: RF, gz)

```
seq.addBlock(rf,gz);
```



Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

1. Hardware limits:

```
sys = mr.opts('MaxGrad', 28, 'GradUnit',
'mT/m','MaxSlew', 120, 'SlewUnit',
'T/m/s', 'rfRingdownTime', 20e-
6, 'rfDeadTime', 100e-6, 'adcDeadTime',
10e-6);
```

2. Create sequence object

```
seq=mr.Sequence(sys);
```

3. Generate elements (e.g. RF, gz)

```
[rf, gz] = mr.makeSincPulse(FlipAngle,
'Duration', 4e-3, 'SliceThickness',
sliceThickness, 'apodization',0.5,
'timeBwProduct', 4, 'system',sys);
```

4. Add blocks (e.g. Block 1: RF, gz)

```
seq.addBlock(rf,gz);
```

Blocks

	duration	rf	gx	gy	gz	adc	ext
1	42	1	0	0	0	0	0
2	500	0	0	0	0	0	0
3	10244	0	0	0	0	1	0

Duration	rf	gx	gy	gz	adc	ext
414	1	0	0	1	0	0

```
>> seq.addBlock(rf,gz)
>> seq

seq =

  Sequence with properties:

          version_major: 1
          version_minor: 5
       version_revision: 1
           rfRasterTime: 1.0000e-06
         gradRasterTime: 1.0000e-05
          adcRasterTime: 1.0000e-07
    blockDurationRaster: 1.0000e-05
            definitions: [4×1 containers.Map]
           blockEvents: {[0 1 0 0 1 0 0]}
       blockDurations: 0.0041
              rfLibrary: [1×1 mr.EventLibrary]
            gradLibrary: [1×1 mr.EventLibrary]
             adcLibrary: [1×1 mr.EventLibrary]
            trigLibrary: [1×1 mr.EventLibrary]
        labelsetLibrary: [1×1 mr.EventLibrary]
        labelincLibrary: [1×1 mr.EventLibrary]
       extensionLibrary: [1×1 mr.EventLibrary]
           shapeLibrary: [1×1 mr.EventLibrary]
          rfShimLibrary: [1×1 mr.EventLibrary]
       softDelayLibrary: [1×1 mr.EventLibrary]
        softDelayHints1: [0×1 containers.Map]
        softDelayHints2: {}
        rotationLibrary: [1×1 mr.EventLibrary]
     extensionStringIDs: {}
     extensionNumericIDs: []
          gradCheckData: [1×1 struct]
          signatureType: ''
          signatureFile: ''
         signatureValue: ''
           rfID2NameMap: [0×1 containers.Map]
          adcID2NameMap: [0×1 containers.Map]
         gradID2NameMap: [0×1 containers.Map]
                    sys: [1×1 struct]

fx >>
```

rf and gz

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

pulseq-master ▸ matlab ▸ +mr ▸

Write *.Seq file

```
# Pulseq sequence file
# Created by MATLAB mr toolbox

[VERSION]
major 1
minor 5
revision 1

[DEFINITIONS]
AdcRasterTime 1e-07
BlockDurationRaster 1e-05
GradientRasterTime 1e-05
RadiofrequencyRasterTime 1e-06

# Format of blocks:
# NUM DUR RF  GX  GY  GZ  ADC  EXT
[BLOCKS]
1 414   1   0   0   1  0  0

# Format of RF events:
# id ampl. mag_id phase_id time_shape_id center delay freqPPM phasePPM freq phase use
# ..   Hz      ..       ..            ..     us    us     ppm  rad/MHz   Hz   rad  ..
# Field 'use' is the initial of:
#   excitation refocusing inversion saturation preparation other undefined
[RF]
1       41.144 1 2 0 2000 100 0 0 0 0 e

# Format of trapezoid gradients:
# id amplitude rise flat fall delay
# ..      Hz/m   us   us   us    us
[TRAP]
 1       200000  40 4000  40  60
```

```
signatureFile: ''
signatureValue: ''
r   write        [0×1 c
ad  write_file   [0×1 c
gra write_v141   [0×1 c
    writeBinary  [1×1 s
>> seq.write
```

```
>> seq.write('Sequence_name.seq')
>>
```

Pulseq: *.seq file

Design the MR pulse sequence

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io

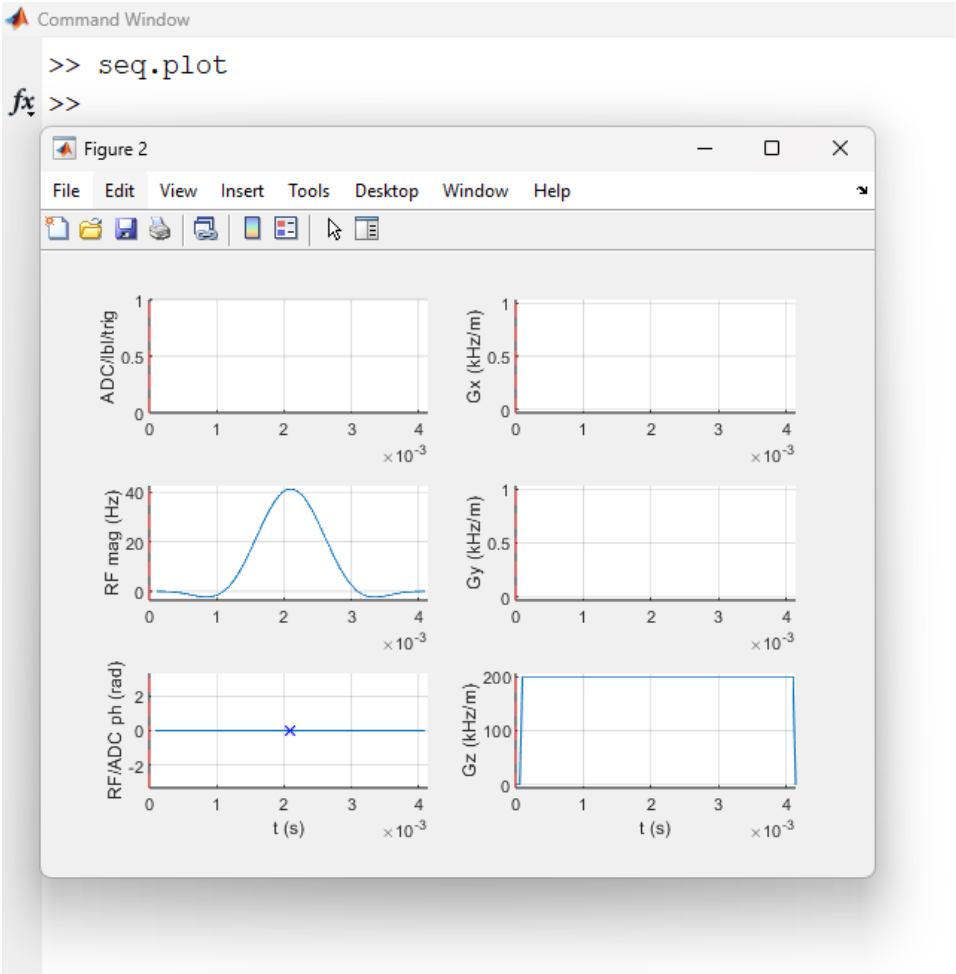
pulseq-master ▸ matlab ▸ +mr ▸

```
>> seq.setDefinition('CreationDate', sprintf('%s',string(datetime)));
>> seq.setDefinition('Name', 'Sequence_name.seq');
>> seq.write('Sequence_name.seq')
>>
```

Write Sequence_name.seq file

```
# Pulseq sequence file
# Created by MATLAB mr toolbox

[VERSION]
major 1
minor 5
revision 1

[DEFINITIONS]
AdcRasterTime 1e-07
BlockDurationRaster 1e-05
CreationDate 19-Feb-2026 17:21:21
GradientRasterTime 1e-05
Name Sequence_name.seq
RadiofrequencyRasterTime 1e-06

# Format of blocks:
# NUM DUR RF  GX  GY  GZ  ADC  EXT
[BLOCKS]
1 414   1   0   0   1   0   0
```

Pulseq: seq.plot()



Pulseq: Main steps

1. Hardware limits

```
% Set system limits
sys = mr.opts('MaxGrad',28,'GradUnit','mT/m',...
    'MaxSlew',150,'SlewUnit','T/m/s','rfRingdownTime', 20e-6, ...
    'rfDeadtime', 100e-6, 'adcDeadTime', 10e-6, 'B0', 3 );
```

2. Acquisition parameters

```
fov=200e-3; Nx=128; Ny=Nx; % Define FOV and resolution
alpha=15;                  % flip angle
thickness=5e-3;            % slice
Nslices=1;
TR=10e-3;
TE=4.3e-3;
```

3. Define events (ADC, gradient, rf, etc…)

```
deltak=1/fov;
gx = mr.makeTrapezoid('x','FlatArea',Nx*deltak,'FlatTime',roDuration,'system',sys);
adc = mr.makeAdc(Nx,'Duration',gx.flatTime,'Delay',gx.riseTime,'system',sys);
```

4. Define delays events (according with TR, TE, etc)

Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

RF

```
% Create alpha-degree slice selection pulse and gradient
[rf, gz] = mr.makeSincPulse(alpha*pi/180,'Duration',3e-3,...
    'SliceThickness',thickness,'apodization',0.42,...
    'timeBwProduct',4,'use','excitation','system',sys);
```

```
>> rf

rf =

  struct with fields:

           type: 'rf'
         signal: [3000×1 double]
              t: [3000×1 double]
      shape_dur: 0.0030
     freqOffset: 0
    phaseOffset: 0
        freqPPM: 0
      phasePPM: 0
       deadTime: 1.0000e-04
   ringdownTime: 2.0000e-05
          delay: 1.0000e-04
            use: 'excitation'
         center: 0.0015

>>
```

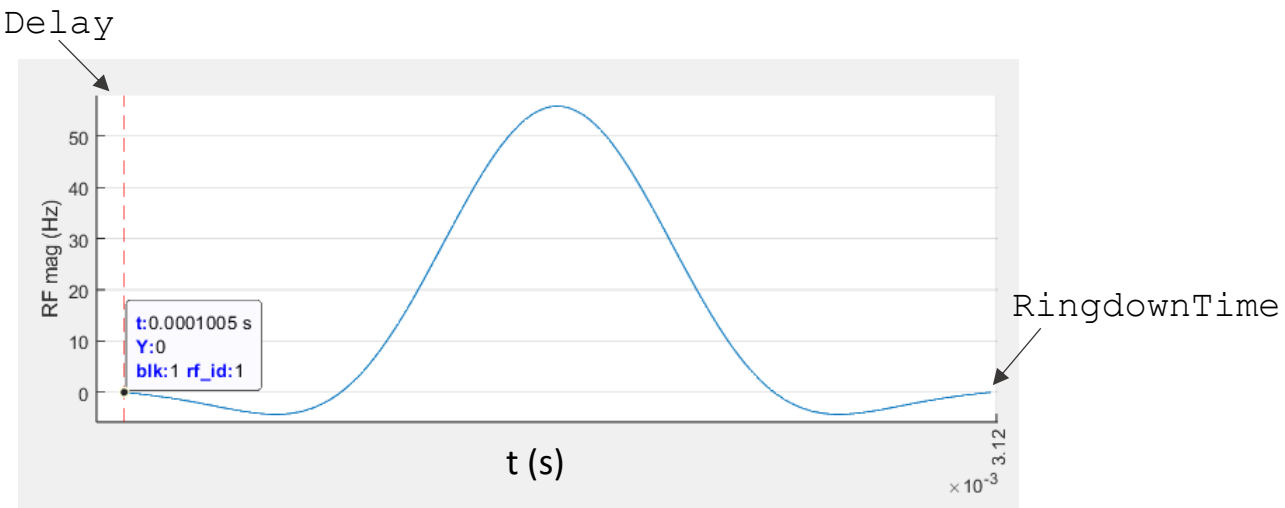
Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

RF

```
% Create alpha-degree slice selection pulse and gradient
[rf, gz] = mr.makeSincPulse(alpha*pi/180,'Duration',3e-3,...
    'SliceThickness',thickness,'apodization',0.42,...
    'timeBwProduct',4,'use','excitation','system',sys);
```

```
>> rf

rf =

  struct with fields:

           type: 'rf'
         signal: [3000×1 double]
              t: [3000×1 double]
      shape_dur: 0.0030
     freqOffset: 0
    phaseOffset: 0
        freqPPM: 0
      phasePPM: 0
       deadTime: 1.0000e-04
   ringdownTime: 2.0000e-05
          delay: 1.0000e-04
            use: 'excitation'
         center: 0.0015

>>
```




```
>> totalDuration_ms = mr.calcDuration(rf)*1000

totalDuration_ms =

    3.1200
```

Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

RF

```
% Create alpha-degree slice selection pulse and gradient
[rf, gz] = mr.makeSincPulse(alpha*pi/180,'Duration',3e-3,...
    'SliceThickness',thickness,'apodization',0.42,...
    'timeBwProduct',4,'use','excitation','system',sys);
```

```
>> rf

rf =

  struct with fields:

           type: 'rf'
         signal: [3000×1 double]
              t: [3000×1 double]
      shape_dur: 0.0030
     freqOffset: 0
    phaseOffset: 0
        freqPPM: 0
      phasePPM: 0
       deadTime: 1.0000e-04
   ringdownTime: 2.0000e-05
          delay: 1.0000e-04
            use: 'excitation'
         center: 0.0015

>>
```

```
>> validPulseUses = mr.getSupportedRfUse()

validPulseUses =

  1×7 cell array

  {'excitation'}  {'refocusing'}  {'inversion'}  {'saturation'}  {'preparation'}  {'other'}  {'undefined'}
```

Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

RF

```
% Refocusing pulse with spoiling gradients
rf180 = mr.makeBlockPulse(pi,sys,'Duration',500e-6,'use','refocusing');
gzSpoil = mr.makeTrapezoid('z',sys,'Area',gz.area*2,'Duration',3*preTime);
```

```
>> validPulseUses = mr.getSupportedRfUse()

validPulseUses =

  1×7 cell array

    {'excitation'}    {'refocusing'}    {'inversion'}    {'saturation'}    {'preparation'}    {'other'}    {'undefined'}
```

Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

RF

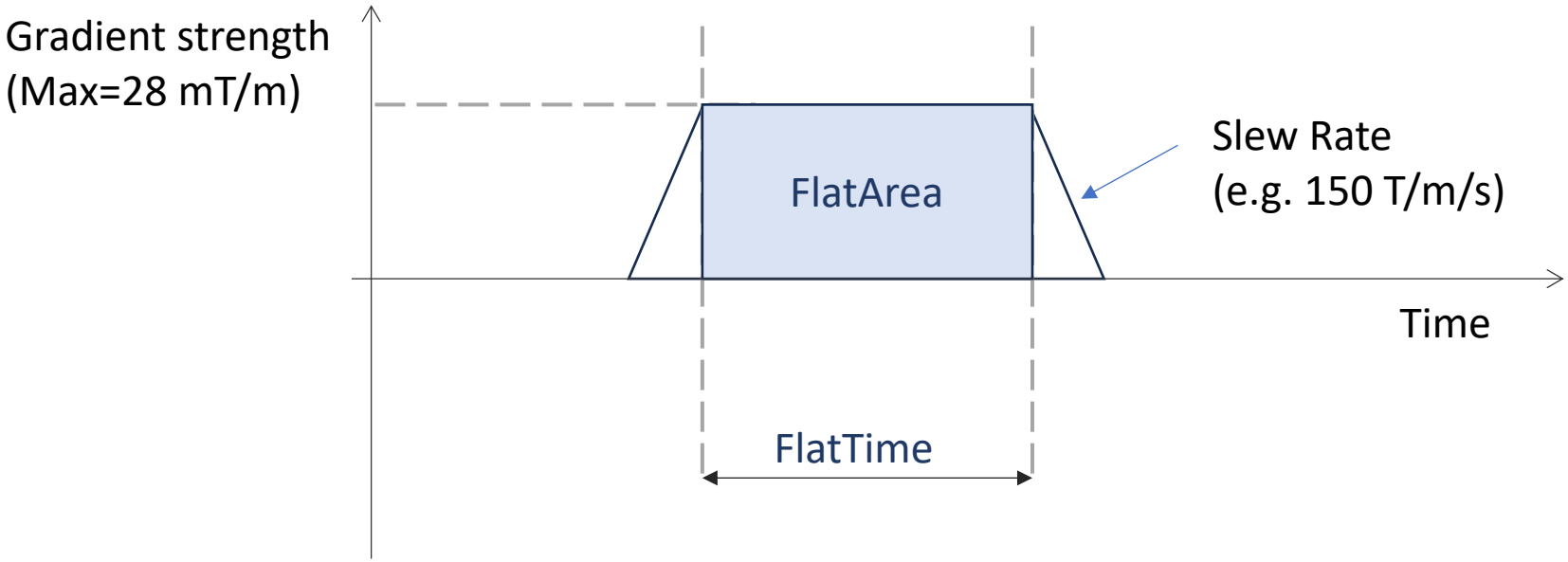
```
>> mr.makeAdiabaticPulse

>> mr.makeGaussPulse

>> mr.makeSLRpulse

>> mr.makeArbitraryRf
```

Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

Gradients

```
deltak=1/fov;
gx = mr.makeTrapezoid('x','FlatArea',Nx*deltak,'FlatTime',roDuration,'system',sys);
```



Pulseq: Events

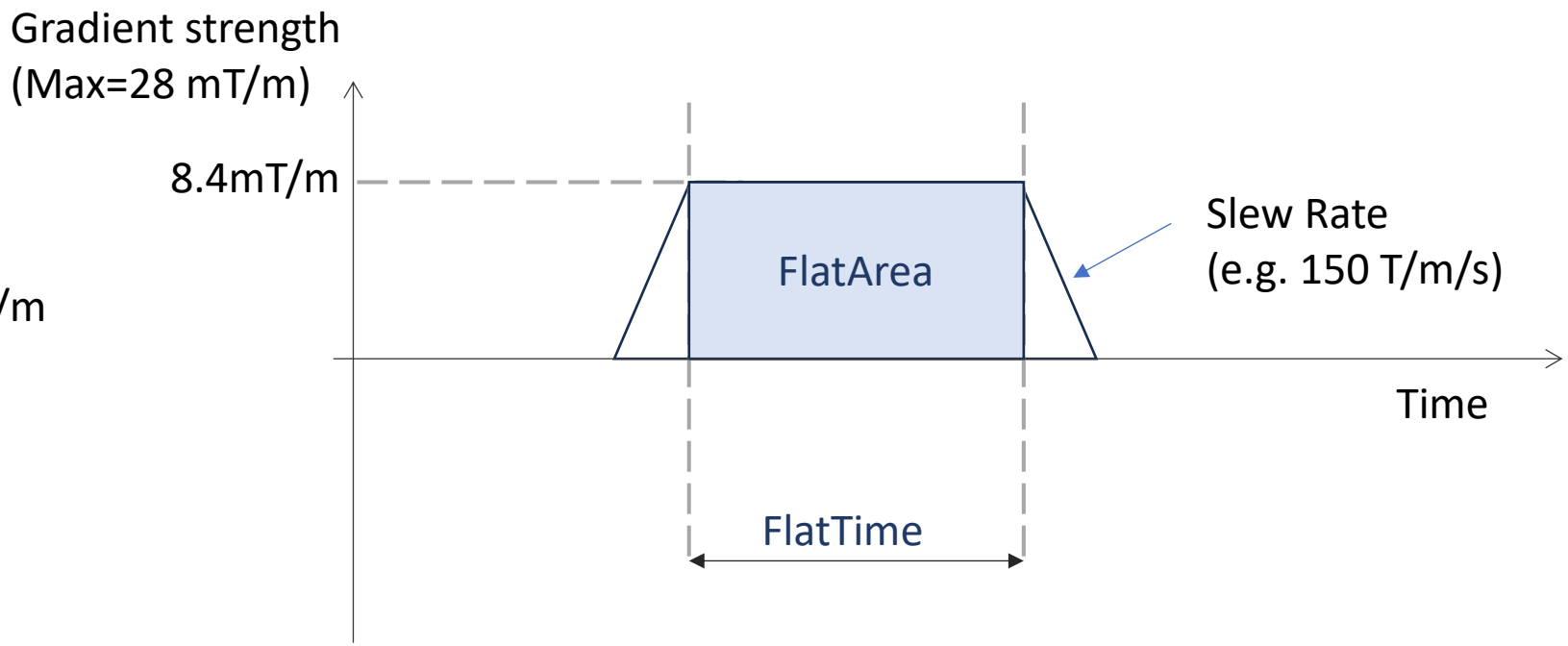
3. Define events (ADC, gradient, rf, etc…)

Gradients

```
deltak=1/fov;
gx = mr.makeTrapezoid('x','FlatArea',Nx*deltak,'FlatTime',roDuration,'system',sys);
```

```
>> gx

gx =

  struct with fields:

        type: 'trap'
     channel: 'x'
   amplitude: 3.5714e+05
    riseTime: 6.0000e-05
    flatTime: 0.0032
    fallTime: 6.0000e-05
        area: 1.1643e+03
    flatArea: 1.1429e+03
       delay: 0
       first: 0
        last: 0
```

Hz/m = 8.4mT/m



Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

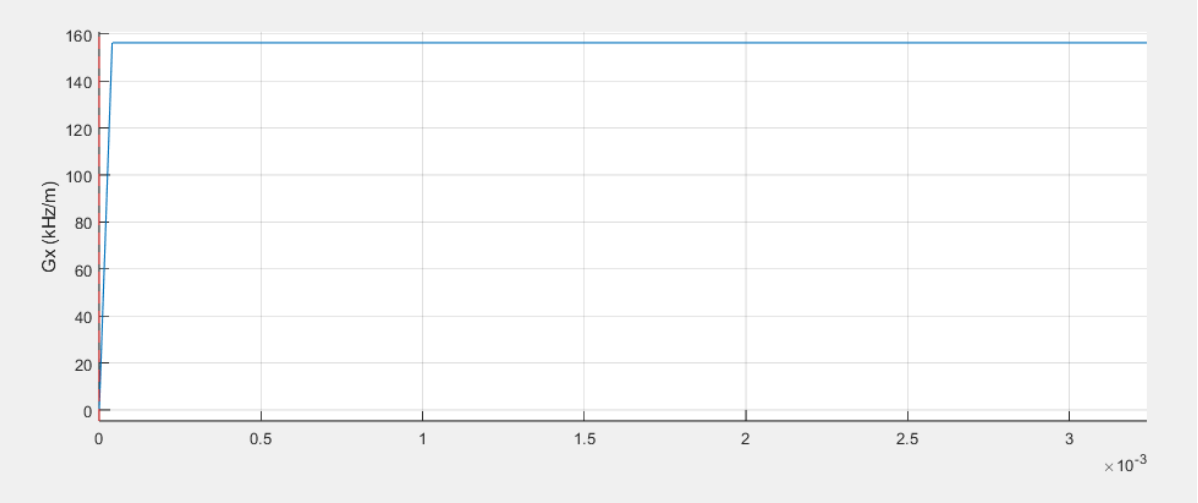
Gradients

Cut the RO gradient into two parts

```
gx_1=mr.splitGradientAt(gx, gx.riseTime+gx.flatTime)
```

gx_1(1)

```
>> gx_1(1)

ans =

  struct with fields:

        type: 'grad'
     channel: 'x'
    waveform: [3×1 double]
       delay: 0
          tt: [3×1 double]
   shape_dur: 0.0032
        area: 503.1250
       first: 0
        last: 156250
```



Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

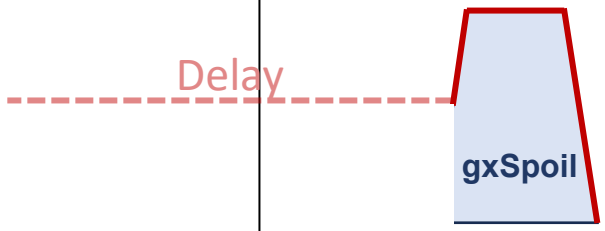
Gradients

```
gxSpoil = mr.makeExtendedTrapezoidArea('x',
gx.amplitude,0,2*Ny*deltak, sys)
```

```
gxSpoil =

  struct with fields:

        type: 'grad'
     channel: 'x'
    waveform: [4×1 double]
       delay: 0
          tt: [4×1 double]
   shape_dur: 0.0012
        area: 1.0000e+03
       first: 156250
        last: 0
```

```
gxSpoil.delay = mr.calcDuration(gx_1(1))
```

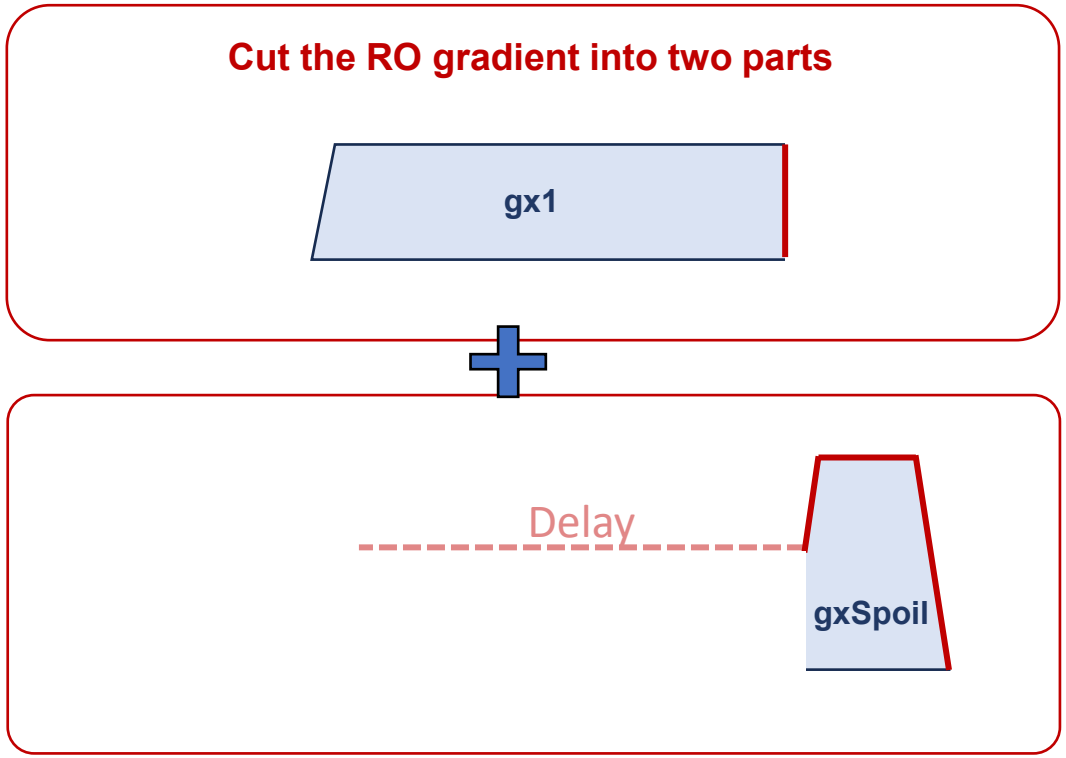
```
>> gxSpoil.delay = mr.calcDuration(gx_1(1))

gxSpoil =

  struct with fields:

        type: 'grad'
     channel: 'x'
    waveform: [4×1 double]
       delay: 0.0032
          tt: [4×1 double]
   shape_dur: 0.0012
        area: 1.0000e+03
       first: 156250
        last: 0
```

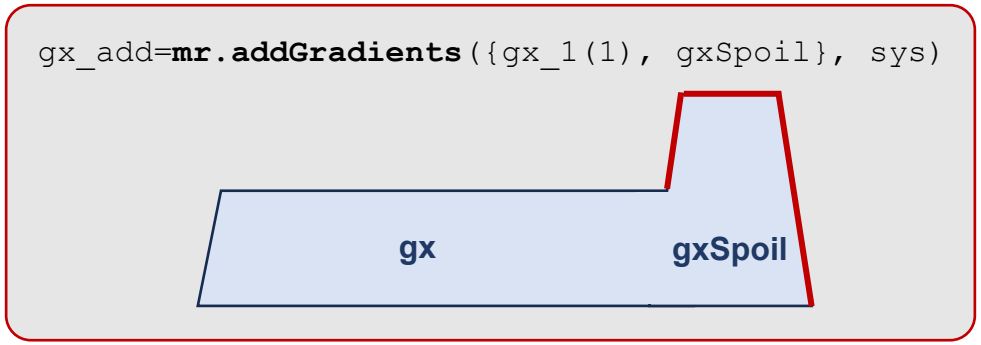


Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

Gradients



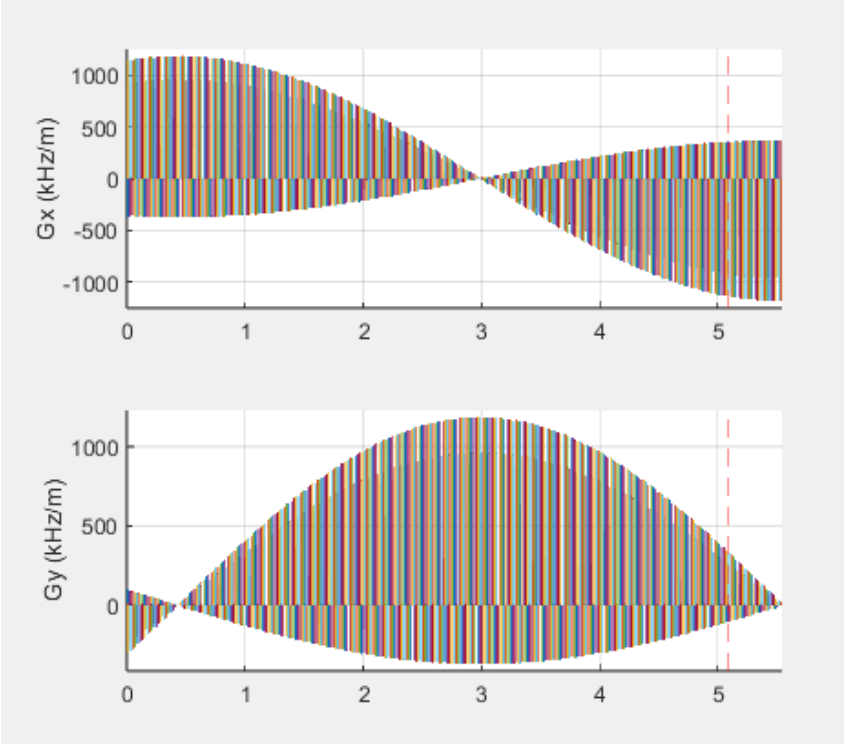



```
struct with fields:

       type: 'grad'
    channel: 'x'
   waveform: [6×1 double]
      delay: 0
         tt: [6×1 double]
  shape_dur: 0.0045
       area: 1.5031e+03
      first: 0
       last: 0
```

Pulseq: Events

3. Define events (ADC, gradient, rf, etc…)

Gradients



Rotate gradients
(e.g. radial acquisitions)

```
seq.addBlock(mr.rotate('z',phi,gxPre,gzReph));
```

Scale gradients

```
gx = mr.scaleGrad(gx, -1) ;
```

Align events

```
[rf,~]=mr.align('right',rf,gz_1);
```

Trajectory to gradient

```
[ga, sa] = mr.traj2grad(
```

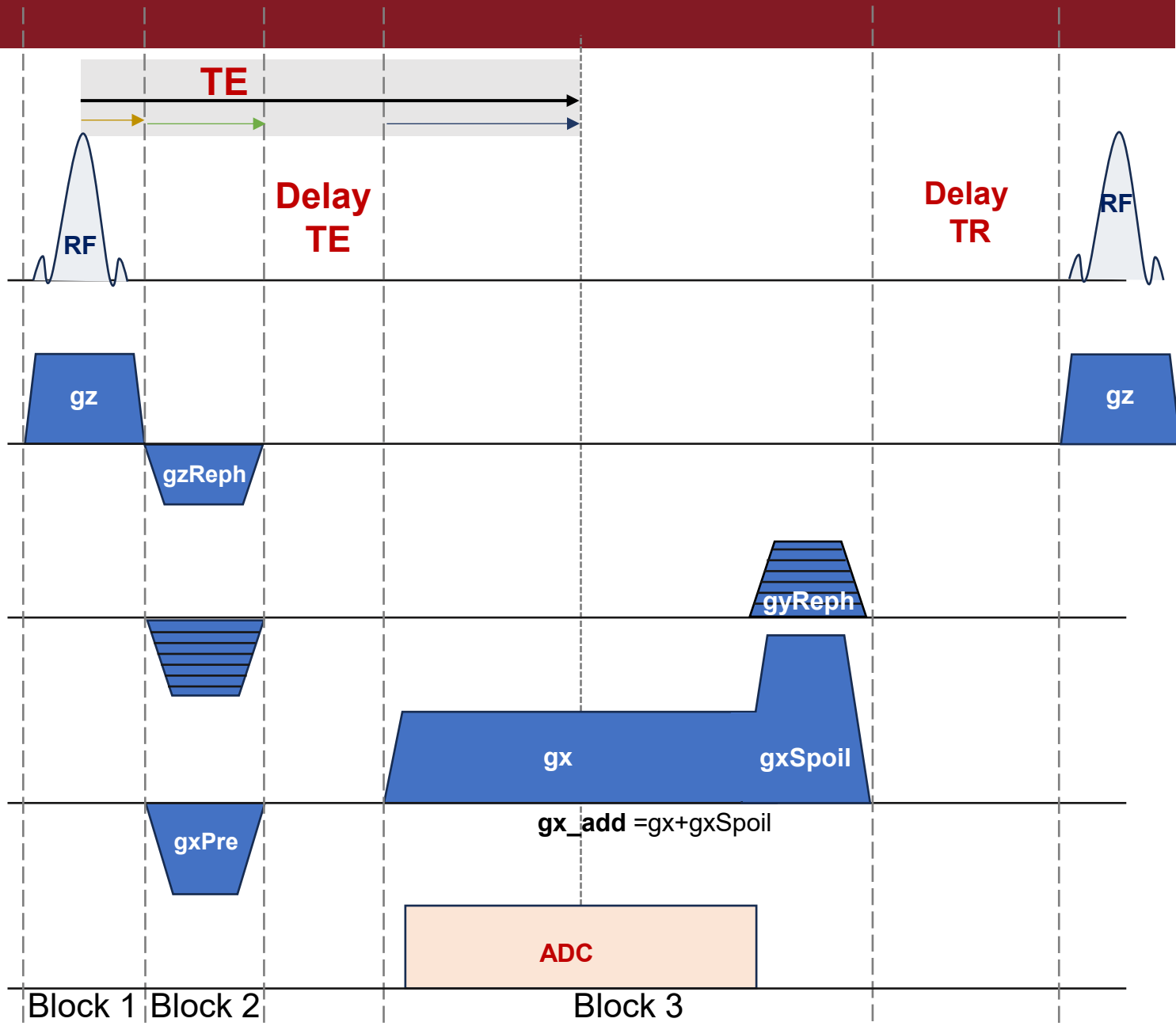
Pulseq: Delays

4. Define delays events (according with TR, TE, etc)

Delays

Delay TR

```
% Calculate timing
delayTE=ceil((TE - mr.calcDuration(gxPre) ...
    - gz.fallTime - gz.flatTime/2 ...
    - mr.calcDuration(gx)/2)/seq.gradRasterTime)...
    *seq.gradRasterTime;
```



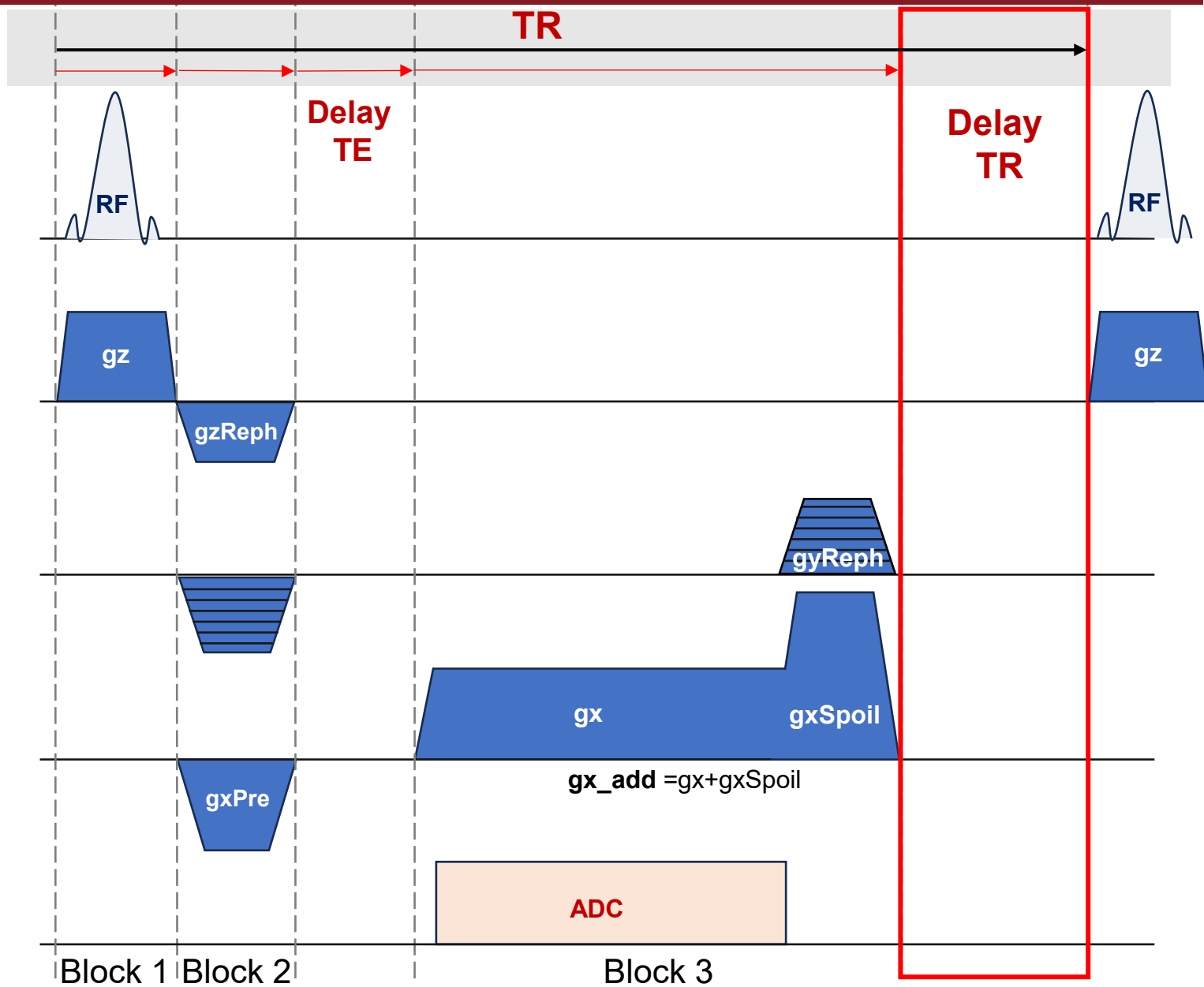
Pulseq:Delays

4. Define delays events (according with TR, TE, etc)

Delays

Delay TR

```
delayTR=ceil((TR - mr.calcDuration(gz) -...
    mr.calcDuration(gxPre) ...
    - mr.calcDuration(gx) - delayTE)...
    /seq.gradRasterTime)*seq.gradRasterTime;
```

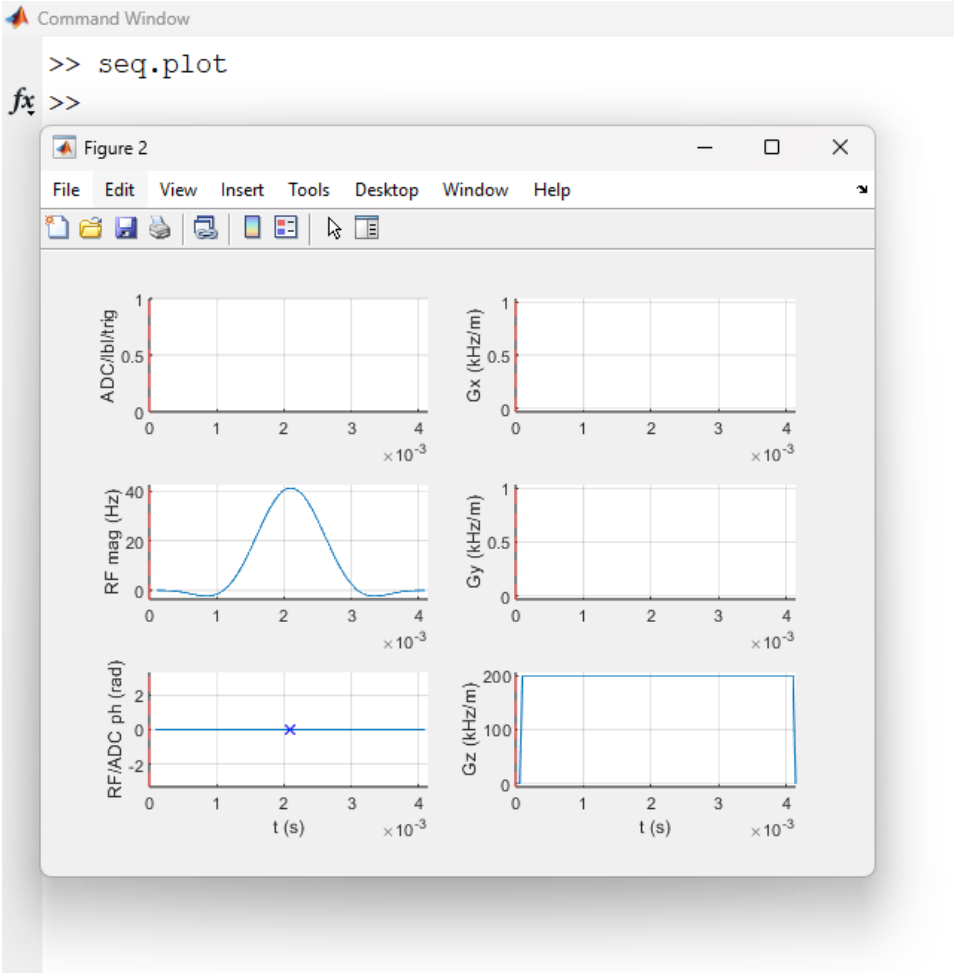


Pulseq: Add blocks

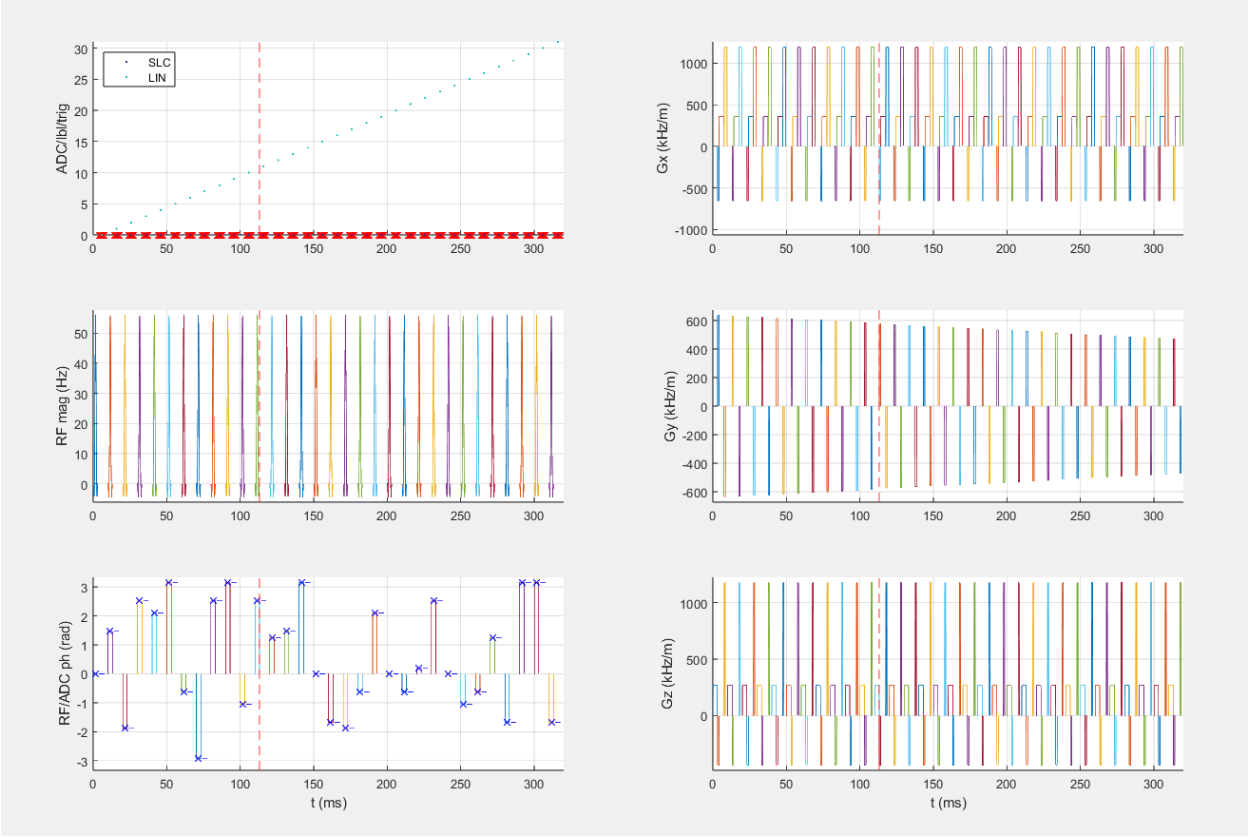
5. Add blocks

```
% loop over slices
for s=1:Nslices
    % loop over phase encodes and define sequence blocks
    for i=1:Ny
      for c=1:length(TE)
        seq.addBlock(rf,gz);
        gyPre = mr.makeTrapezoid('y','Area',phaseAreas(i),...
            'Duration',mr.calcDuration(gxPre),'system',sys);
        seq.addBlock(gxPre,gyPre,gzReph);
        seq.addBlock(mr.makeDelay(delayTE(c)));
        seq.addBlock(gx,adc);
```

Pulseq: seq.plot()

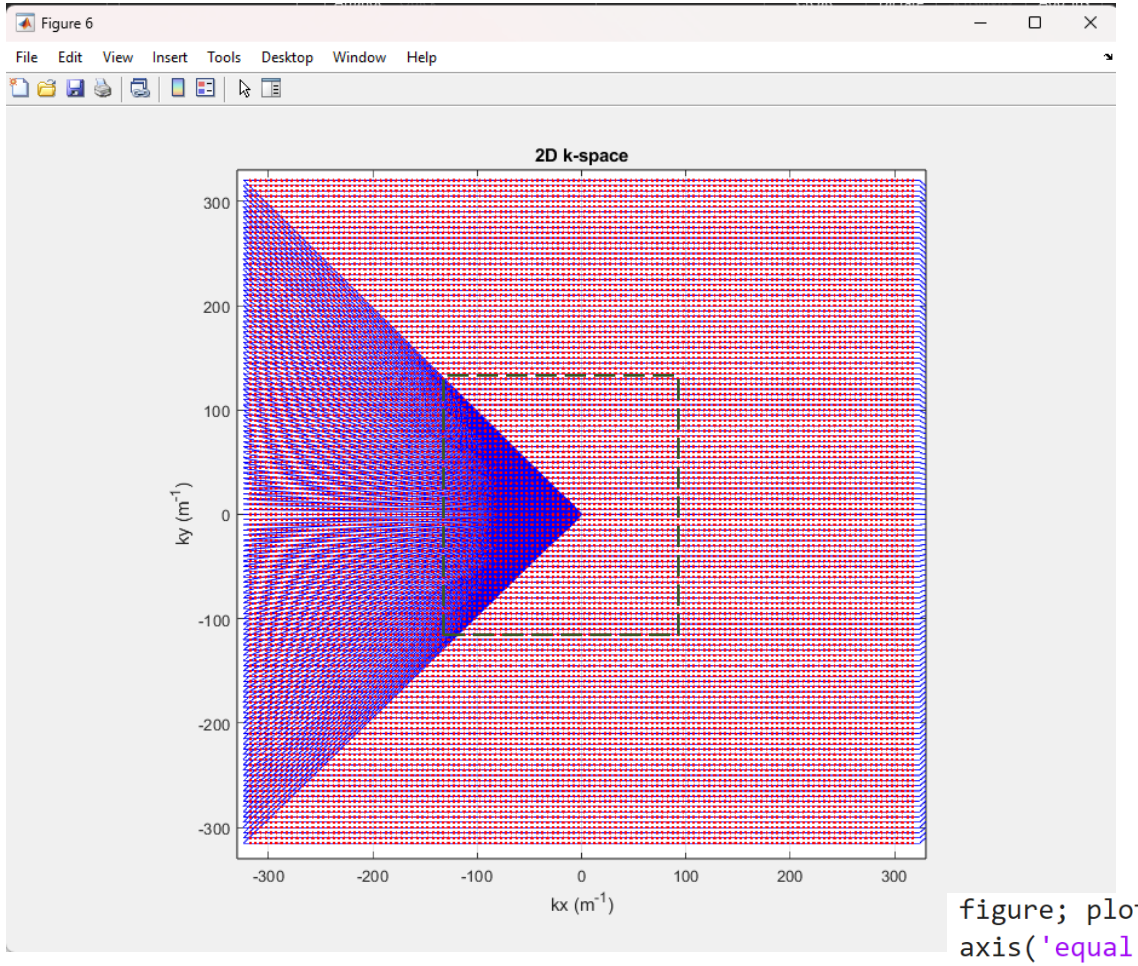


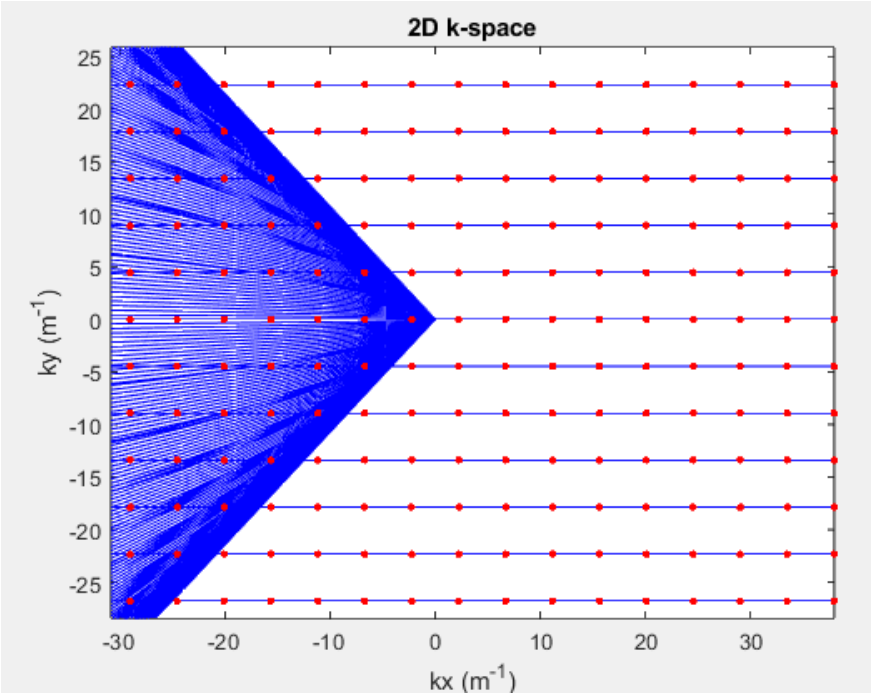

```
seq.plot('timeRange', [0 32]*TR, 'TimeDisp', 'ms', 'Label', 'LIN,SLC');
```



Pulseq: k-space trajectory

```
% k-space trajectory calculation
[ktraj_adc, t_adc, ktraj, t_ktraj, t_excitation, t_refocusing] = seq.calculateKspacePP();
```






```
figure; plot(ktraj(1,:),ktraj(2,:),'b'); % a 2D plot
axis('equal'); % enforce aspect ratio for the correct trajectory display
hold;plot(ktraj_adc(1,:),ktraj_adc(2,:),'r.', 'MarkerSize',10); % plot the sampling points
title('2D k-space');
```

Pulseq: seq.testRepot

Test *.seq

```
>> rep = seq.testReport;
fprintf([rep{:}])
Number of blocks: 1287
Number of events:
   RF:      256
   Gx:      768
   Gy:      512
   Gz:      768
   ADC:     256
Sequence duration: 12.560000s
TE: 0.004300s
TR: 0.010000s
Flip angle: 15.00°
Unique k-space positions (a.k.a. columns, rows, etc): 128
Unique k-space positions (a.k.a. columns, rows, etc): 128
Dimensions: 2
   Spatial resolution: 1.57 mm
   Spatial resolution: 1.56 mm
Repetitions/slices/contrasts: 2  range: [2 2]
   16384 k-space position(s) repeated 2 times
Cartesian encoding trajectory detected
Block timing check passed successfully
Max. Gradient: 1185185 Hz/m == 27.84 mT/m
Max. Gradient: 340426 Hz/m == 8.00 mT/m
Max. Gradient: 1176471 Hz/m == 27.63 mT/m
Max. Slew Rate: 6.23782e+09 Hz/m/s == 146.51 T/m/s
Max. Slew Rate: 6.38021e+09 Hz/m/s == 149.85 T/m/s
Max. Slew Rate: 6.2468e+09 Hz/m/s == 146.72 T/m/s
Max. Absolute Gradient: 1704299 Hz/m == 40.03 mT/m
Max. Absolute Slew Rate: 1.08608e+10 Hz/m/s == 255.09 T/m/s
```

Pulseq

Test *.seq

Command Window

```
>> [ok, error_report]=seq.checkTiming

ok =

  logical

   1


error_report =

  0×0 empty cell array

>>
```

Pulseq: Other useful commands

Task	Command
Arbitrary gradients e.g. k-space trajectories	**makeArbitraryRf** or **makeArbitraryGrad**
Trigger event for a synchronisation with an external signal	trig_crush=**mr.makeTrigger**('physio1','duration', Tdelay_trig, 'delay', 100e-6);
Testing and Checktimes	[report] = **seq. testReport;** [ok, error_report]=**seq.checkTiming**;
SAR	[total_energy, peak_pwr, rf_rms]=**seq.calcRfPower**;
PNS	[ok, pns_norm, pns_comp, t_axis]=**seq.calcPNS**(*.asc)
Read sequence	**seq.read**('filename');

Pulseq: Where to start

Pulseq

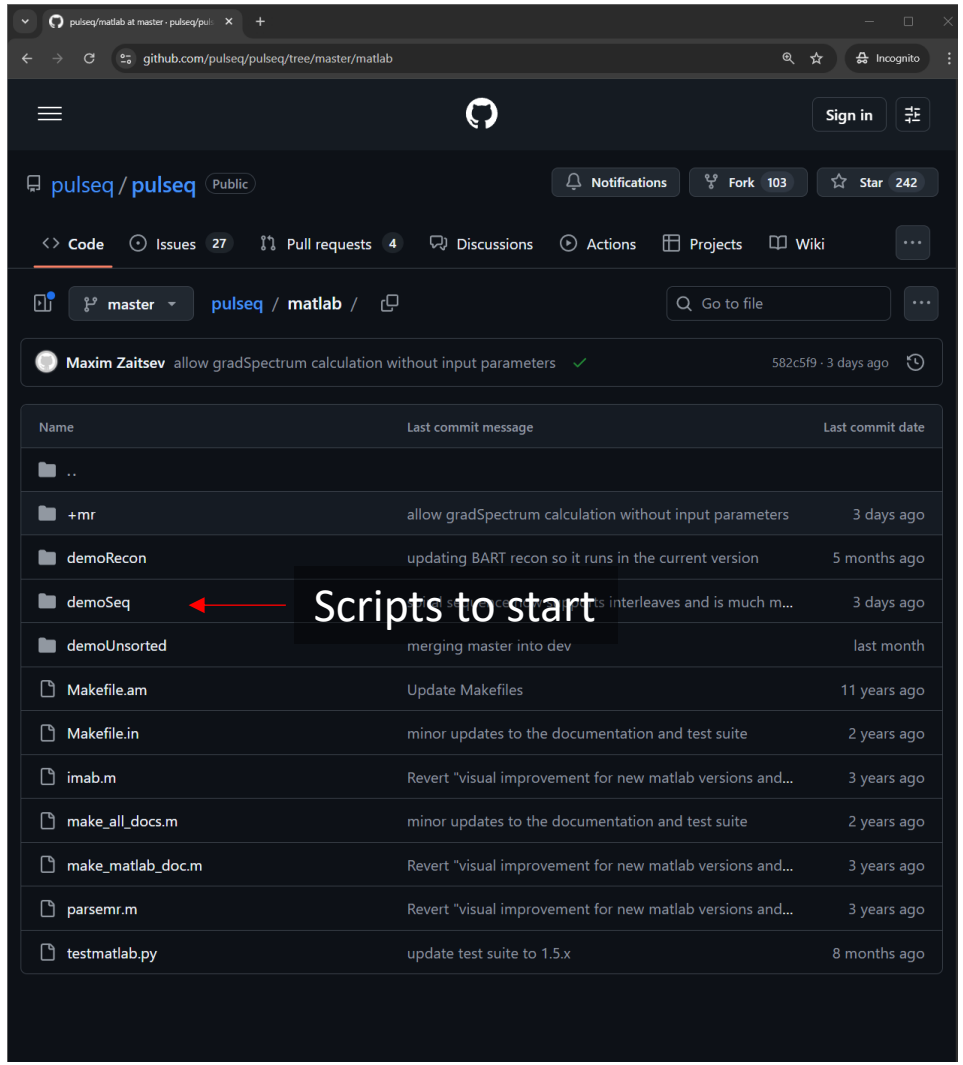
Demos

MATLAB

Pulseq

Layton KJ, et al. *MRM*. 2017

https://pulseq.github.io



Thank you all!

Rita G Nunes, UL
Teresa Correia, CCMAR
Laseeb colleagues